

Multi-File Topology Design and Enterprise Services

Juan David Guarnizo Hernandez, PhD
juan.guarnizo@tec.mx

TC2006B: Device Communications
Tecnológico de Monterrey
FJ2026

Goals

By the end of this comprehensive exercise you will:

1. Understand how to encapsulate network zones into modular Python classes (OOP).
2. Integrate multiple independent sub-topologies into a unified Wide Area Network (WAN) core.
3. Configure Equal-Cost Multi-Path (ECMP) static routing to achieve load balancing across a diamond WAN topology.
4. Deploy, expose, and verify enterprise IT services (HTTP, FTP) across the emulated network.

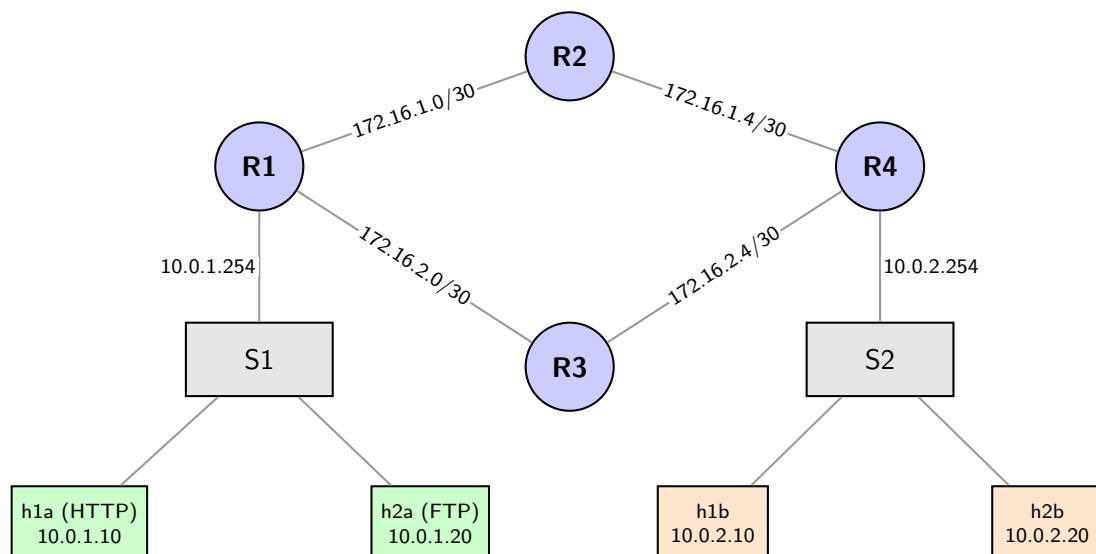


Figure 1: Target Architecture: Multi-path WAN core connecting two distributed branch sites.

0) Pre-requirements & Software Installation

As your Mininet topologies grow in complexity, writing them into a single monolithic Python file becomes unmanageable. In this lab, we will break the network into modular Python objects representing geographic sites, and connect them together using a master script.

We also require specific Python libraries to easily spin up enterprise application services (HTTP, FTP). Run the following commands:

```
1 sudo apt-get update
2 sudo apt-get install -y mininet net-tools iputils-ping tcpdump wireshark
  curl ftp
3 sudo apt-get install -y python3-pyftplib
```

1) Modular Topology: The Access Networks

First, we will define the internal LAN of “Network A”. Instead of initializing Mininet immediately, we define a Python class that accepts a global Mininet net object and builds the components locally.

Create a file named `site_a.py` and add the following code:

```
1 from mininet.node import Node
2
3 # Custom Router Class to enable routing and disable Reverse Path (RP)
  Filtering
4 class Router(Node):
5     def config(self, **params):
6         super(Router, self).config(**params)
7         self.cmd('sysctl -w net.ipv4.ip_forward=1')
8         self.cmd('sysctl -w net.ipv4.conf.all.rp_filter=0')
9         self.cmd('sysctl -w net.ipv4.conf.default.rp_filter=0')
10
11     def terminate(self):
12         self.cmd('sysctl -w net.ipv4.ip_forward=0')
13         super(Router, self).terminate()
14
15 class SiteA:
16     def __init__(self):
17         self.gateway = None
18
19     def build(self, net):
20         self.gateway = net.addHost('r1', cls=Router, ip='10.0.1.254/24')
21
22         s1 = net.addSwitch('s1', failMode='standalone')
23         h1a = net.addHost('h1a', ip='10.0.1.10/24', defaultRoute='via
10.0.1.254')
24         h2a = net.addHost('h2a', ip='10.0.1.20/24', defaultRoute='via
10.0.1.254')
25
26         net.addLink(h1a, s1)
27         net.addLink(h2a, s1)
28         net.addLink(s1, self.gateway)
```

Student Task: Create Site B

Using the architecture from Figure 1, create a matching `site_b.py` file containing a `SiteB` class. It must deploy `r4`, `s2`, `h1b`, and `h2b` using the `10.0.2.0/24` subnet. Keep the Router class included at the top of your file.

2) The Master Core and ECMP Routing

We will now create a master script that imports your site classes, builds the core WAN routers (r2 and r3), connects the geographic boundaries, and applies advanced static routing.

Notice the use of Equal-Cost Multi-Path (ECMP) routing using the `nexthop` command. This allows the edge routers to load balance traffic across both core links simultaneously.

Create `master_wan.py`:

```
1#!/usr/bin/env python3
2from mininet.net import Mininet
3from mininet.node import OVSSwitch
4from mininet.link import TCLink
5from mininet.cli import CLI
6from mininet.log import setLogLevel
7
8# Import our modular site topologies
9from site_a import SiteA, Router
10from site_b import SiteB
11
12def build_master():
13    net = Mininet(controller=None, switch=OVSSwitch, link=TCLink)
14
15    site_a = SiteA()
16    site_b = SiteB()
17    site_a.build(net)
18    site_b.build(net)
19
20    r2 = net.addHost('r2', cls=Router)
21    r3 = net.addHost('r3', cls=Router)
22
23    net.addLink(site_a.gateway, r2)
24    net.addLink(site_a.gateway, r3)
25    net.addLink(site_b.gateway, r2)
26    net.addLink(site_b.gateway, r3)
27
28    net.start()
29    r1, r4 = site_a.gateway, site_b.gateway
30
31    r1.setIP('172.16.1.1/30', intf='r1-eth1')
32    r1.setIP('172.16.2.1/30', intf='r1-eth2')
33    r4.setIP('172.16.1.6/30', intf='r4-eth1')
34    r4.setIP('172.16.2.6/30', intf='r4-eth2')
35
36    r2.setIP('172.16.1.2/30', intf='r2-eth0')
37    r2.setIP('172.16.1.5/30', intf='r2-eth1')
38    r3.setIP('172.16.2.2/30', intf='r3-eth0')
39    r3.setIP('172.16.2.5/30', intf='r3-eth1')
40
41    r2.cmd('ip route replace 10.0.1.0/24 nexthop via 172.16.1.1 dev r2-eth0')
42    r2.cmd('ip route replace 10.0.2.0/24 nexthop via 172.16.1.6 dev r2-eth1')
43    r3.cmd('ip route replace 10.0.1.0/24 nexthop via 172.16.2.1 dev r3-eth0')
44    r3.cmd('ip route replace 10.0.2.0/24 nexthop via 172.16.2.6 dev r3-eth1')
45
46    r1.cmd('ip route replace 10.0.2.0/24 nexthop via 172.16.1.2 dev r1-eth1')
```

```

47     nexthop via 172.16.2.2 dev r1-eth2')
48     r4.cmd('ip route replace 10.0.1.0/24 nexthop via 172.16.1.5 dev r4-eth1
49     nexthop via 172.16.2.5 dev r4-eth2')
50
51     CLI(net)
52     net.stop()
53
54 if __name__ == '__main__':
55     setLogLevel('info')
56     build_master()

```

Execute your network: `sudo python3 master_wan.py`.

Verify reachability by running `h1a ping -c 3 h1b` inside the Mininet CLI.

3) Deploying Enterprise Services

With end-to-end connectivity established over the WAN, we will now emulate physical servers hosting services in Network A, and client machines accessing them from Network B.

Deploying an HTTP Web Server (h1a)

Open an xterm window for h1a. Inside the terminal, configure and launch Python's built-in web server binding to port 80:

```

1 mkdir -p /tmp/web
2 echo "<h1>Welcome to Site A Datacenter</h1>" > /tmp/web/index.html
3 cd /tmp/web
4 python3 -m http.server 80

```

Deploying an FTP File Server (h2a)

Open an xterm window for h2a. Inside the terminal, start the Python FTP library, allowing write access ('-w') on port 21, and configuring a specific username and password:

```

1 mkdir -p /tmp/ftp_share
2 cd /tmp/ftp_share
3 python3 -m pyftplib -p 21 -w -u admin -P secret123

```

Client Access Testing from Site B

To test the services, open two new xterm windows from the Mininet CLI for the client hosts in Site B:

```

1 mininet> xterm h1b h2b

```

1. Test Web Connectivity: Inside the h1b terminal, run:

```

1 # curl -s http://10.0.1.10

```

You should see the HTML payload generated in Site A.

2. Test FTP Interaction: Inside the h2b terminal, run:

```

1 # ftp 10.0.1.20

```

Login using the username `admin` and password `secret123`. Try issuing commands like `ls` to view the directory contents, then type `quit`.

4) Traffic Capture and Protocol Analysis

To deeply understand how these protocols behave on the wire, we will capture and inspect the traffic using `tcpdump` and Wireshark.

Observing TTL Decrements (ICMP)

Every time an IP packet passes through a router, its Time To Live (TTL) is reduced by 1 to prevent infinite routing loops.

Open an xterm for h1b and start listening for incoming ICMP packets on its local interface:

```
# tcpdump -i h1b-eth0 -n -v icmp
```

Open an xterm for h1a and send a single ping:

```
# ping -c 1 10.0.2.10
```

In the h1b terminal, observe the TTL value of the arriving packet. Stop the capture using `Ctrl+C`.

Analyzing HTTP and FTP with Wireshark

We will now capture the application traffic for deep analysis. Open an xterm for h1b and start a packet capture.

Note: Do not use the `&` operator to background the process; leave this terminal window open and running.

```
# tcpdump -i h1b-eth0 -w /tmp/services.pcap
```

While the capture is running, open a second xterm for h1b and repeat the HTTP (`curl`) and FTP queries from Section 3. Make sure to log into the FTP server using the credentials you configured, run an `ls` command, and quit.

Once complete, return to the first h1b terminal and stop the `tcpdump` process using `Ctrl+C`. You can now open the resulting `/tmp/services.pcap` file in Wireshark on your host OS GUI.

Link Failure and Static Routing Limitations

Equal-Cost Multi-Path (ECMP) static routing attempts to balance traffic across both core routers (`r2` and `r3`). Let's observe what happens when a physical link fails in a statically routed environment without a dynamic routing protocol.

Open an xterm for h1a and start a continuous ping to h1b:

```
# ping 10.0.2.10
```

While the ping is running in the xterm, return to the main Mininet CLI and simulate a physical cable cut by bringing down the link between `r1` and `r2`:

```
mininet> link r1 r2 down
```

Carefully observe the ICMP sequence numbers and the resulting packet loss in the h1a xterm. Once you have documented the behavior, bring the link back up (`link r1 r2 up`) in the Mininet CLI and stop the ping.

4) Traffic Capture and Protocol Analysis

To deeply understand how these protocols behave on the wire, we will capture and inspect the traffic using `tcpdump` and Wireshark.

Observing TTL Decrements (ICMP)

Every time an IP packet passes through a router, its Time To Live (TTL) is reduced by 1 to prevent infinite routing loops.

Open an xterm for h1b and start listening for incoming ICMP packets on its local interface:

```
# tcpdump -i h1b-eth0 -n -v icmp
```

From your main Mininet CLI (or an xterm on h1a), send a single ping: `h1a ping -c 1 10.0.2.10`. In the h1b terminal, observe the TTL value of the arriving packet. Stop the capture using `Ctrl+C`.

Analyzing HTTP and FTP with Wireshark

We will now capture the application traffic for deep analysis. Open an xterm for h1b and start a packet capture.

Note: Do not use the `&` operator to background the process; leave this terminal window open and running.

```
# tcpdump -i h1b-eth0 -w /tmp/services.pcap
```

While the capture is running, open a second xterm for h1b (or use the Mininet CLI) and repeat the HTTP (`curl`) and FTP queries from Section 3. Make sure to log into the FTP server using the credentials you configured, run an `ls` command, and quit.

Once complete, return to the first h1b terminal and stop the `tcpdump` process using `Ctrl+C`. You can now open the resulting `/tmp/services.pcap` file in Wireshark on your host OS GUI.

Link Failure and Static Routing Limitations

Equal-Cost Multi-Path (ECMP) static routing attempts to balance traffic across both core routers (r2 and r3). Let's observe what happens when a physical link fails in a statically routed environment without a dynamic routing protocol.

Start a continuous ping in the Mininet CLI:

```
mininet> h1a ping h1b
```

While the ping is running, open another terminal (or use Mininet's secondary prompt if available) and simulate a physical cable cut by bringing down the link between r1 and r2:

```
mininet> link r1 r2 down
```

Carefully observe the ICMP sequence numbers and the resulting packet loss. Once you have documented the behavior, bring the link back up (`link r1 r2 up`) and stop the ping.

5) Reflection Questions

- Object-Oriented Design:** In a massive simulation with 50 distinct city locations, explain why initializing the subnets within Python classes (as we did with `SiteA`) is superior to writing one massive sequential script.
- Kernel Safeguards (RP Filter):** In the custom `Router` class, we explicitly disabled the Linux kernel's Reverse Path (RP) filtering (`rp_filter=0`). In an Equal-Cost Multi-Path (ECMP) routing environment where traffic can travel over one link but return over another (asymmetric routing), what would happen if the RP filter was left enabled?
- WAN Convergence & Failure:** In Section 4.3, you simulated a link failure while utilizing ECMP static routes. Describe the exact behavior of the ICMP traffic (packet loss percentage, pattern of drops). Why does static ECMP fail to seamlessly recover from this outage, and how would a dynamic routing protocol like OSPF solve this issue?

4. **Routing & Time to Live (TTL):** Based on your ICMP packet capture, what was the default initial TTL of the packet leaving the Linux host h1a (you can verify this by pinging localhost), and what was the TTL when it finally arrived at h1b? Based on this math, how many routed hops did the packet take? Does this match your topology diagram?
5. **Application Layer (HTTP):** When inspecting the HTTP traffic in Wireshark, what specific HTTP method was used by the curl command? In which packet did you find the <h1>Welcome to Site A Datacenter</h1> payload? Could you read it in cleartext? What does this imply about the security of standard HTTP?
6. **Application Layer (FTP):** FTP uses two distinct channels: a command channel and a data channel. Look at your FTP capture. On what TCP port was the initial connection established? When you executed the ls command, you should see an Entering Passive Mode response from the server. How did the client and server negotiate a new port for the data transfer, and what was the dynamically assigned port number for the directory listing? Were the FTP login credentials encrypted on the wire?